

Roteiro de Laboratório 1 — Introdução ao OpenMP

Unidade 03 | Continuação da Aula 05 Entrega pelo Moodle ao final da aula.

Objetivos

- Transformar um laço sequencial numa região paralela e reconhecer o efeito da flag `-fopenmp` na compilação e na ligação.
- Controlar o número de threads pelas três formas possíveis e explicar quando usar cada uma.
- Instrumentar, com as próprias mãos, um programa para medir tempo de execução.
- Medir speed-up e eficiência variando o número de threads, e relacionar a curva obtida com as características da própria máquina.

Sobre este roteiro

Os dois programas em `codigo/src/` (`ola_mundo.c`, `conta_primos.c`) são **sequenciais**. Nenhum deles inclui `omp.h`, usa qualquer `#pragma omp` ou mede tempo de execução. Isso é proposital: acrescentar cada uma dessas coisas — a região paralela, a cláusula certa, a medição de tempo — é o próprio exercício. Se você usar uma IA para ajudar a escrever código, tudo bem, mas o objetivo do laboratório é você decidir *onde* cada diretiva entra e *por quê*; não peça para uma IA tomar essas decisões por você.

Preparação

```
cd codigo
make all
```

Só é preciso `gcc` com suporte a OpenMP (padrão em qualquer **build-essential** recente). Deixe um segundo terminal aberto com `htop` (ou o monitor de sistema da sua distribuição) visível durante toda a Parte 3.

Parte 1 — Escrever a primeira região paralela

1. Compile e rode `codigo/src/ola_mundo.c` como ele está:

```
make run-ola_mundo
```

Ele imprime “Ola, mundo!” uma única vez. É sequencial.

2. Edite o arquivo para que o `printf` rode **dentro de uma região paralela**, uma vez por thread, imprimindo o identificador da thread e o total de threads da equipe. Você vai precisar de `#include <omp.h>`, do `#pragma omp parallel` e das funções `omp_get_thread_num()` e `omp_get_num_threads()`. Recompile e rode:

```
make run-ola_mundo
```

Responda: quantas linhas apareceram? De onde vem esse número (você não escreveu nenhum laço nem nenhum “4” em lugar nenhum do código)?

3. Rode de novo, mais de uma vez seguida. **Responda:** a ordem das linhas se repete? Por que o modelo fork-join não garante uma ordem entre as threads?
4. Agora tente compilar o **mesmo arquivo já editado** sem a flag `-fopenmp`:

```
make sem-openmp
```

Responda: o que aconteceu — um aviso, ou um erro que impede a compilação? Se for um erro, ele está na fase de compilação ou na fase de ligação (*linking*)? O `#pragma omp parallel`, sozinho, apenas

vira comentário sem a flag; por que as chamadas a `omp_get_thread_num()` e `omp_get_num_threads()` se comportam de forma diferente do `pragma`?

Parte 2 — Controlar o número de threads

Faça uma cópia do arquivo da Parte 1 para editar livremente:

```
cp src/ola_mundo.c src/ola_mundo_threads.c
```

1. **Pela cláusula.** Acrescente `num_threads(4)` ao `#pragma omp parallel`. Compile e confirme que aparecem exatamente 4 linhas:

```
make run-ola_mundo_threads
```

2. **Pela função de runtime.** Remova a cláusula `num_threads(4)` e, em vez dela, chame `omp_set_num_threads(4)`; antes da região paralela. Recompile e confirme o mesmo resultado.
3. **Pela variável de ambiente.** Sem tocar no código-fonte (nem cláusula, nem chamada de runtime), controle o número de threads de fora:

```
OMP_NUM_THREADS=2 ./bin/ola_mundo_threads
```

Responda:

- Se as três formas estivessem presentes ao mesmo tempo no mesmo programa, qual prevalece? (A Aula 05 já apresentou essa precedência — revise o slide antes de responder por tentativa e erro.)
 - Qual a vantagem de controlar pela variável de ambiente em vez das outras duas formas? E qual a desvantagem — o que ela **não** permite fazer que as outras duas permitem?
4. Acrescente, logo antes da região paralela, uma linha que imprima o número de núcleos da máquina com `omp_get_num_procs()`. Compare o valor com a saída de `nproc` no terminal e com o número de núcleos mostrado pelo `htop`. Eles batem? Se a sua máquina tiver *hyperthreading*/SMT, o que exatamente esse número está contando: núcleos físicos ou lógicos?

Parte 3 — Instrumentar e medir escalabilidade

`codigo/src/conta_primos.c` conta quantos números primos existem abaixo de um limite, testando cada número por divisão. É sequencial e não mede tempo.

1. Compile e rode como está, cronometrando pelo terminal:

```
make run-conta_primos
```

ou, para ver o tempo sem editar nada ainda:

```
time ./bin/conta_primos
```

O valor padrão (10 milhões) foi escolhido para levar alguns segundos numa máquina comum. Se na sua ele passar muito rápido (menos de ~1 s) ou muito devagar (mais de ~15 s), ajuste com `-n` (por exemplo, `./bin/conta_primos -n 20000000`).

2. Acrescente `#include <omp.h>` e meça o tempo da chamada a `conta_primos()` com `omp_get_wtime()` (uma leitura antes, uma depois, e a diferença impressa em segundos). Recompile e confira que o tempo medido internamente bate com o que o `time` do terminal mostrou.
3. Crie uma **segunda** função, `conta_primos_paralelo`, igual à primeira, mas com `#pragma omp parallel for reduction(+:total)` no laço externo. Mantenha a função sequencial original: você vai precisar dela para comparar.
4. Com o `htop` visível, rode o programa variando o número de threads e anote o tempo da versão paralela em cada caso:

```
OMP_NUM_THREADS=1 ./bin/conta_primos
OMP_NUM_THREADS=2 ./bin/conta_primos
OMP_NUM_THREADS=4 ./bin/conta_primos
OMP_NUM_THREADS=8 ./bin/conta_primos
```

(ajuste a lista de threads ao número de núcleos da sua máquina, lido na Parte 2). Observe, no **htop**, quantos núcleos acendem durante a execução com mais threads.

5. Monte sua própria tabela de speed-up e eficiência, usando a versão sequencial (1 thread real, não a versão paralela rodando com `OMP_NUM_THREADS=1`) como referência de tempo:

Threads	Tempo paralelo (s)	Speed-up (t_{seq}/t_N)	Eficiência ($speed-up/N$)
---------	--------------------	----------------------------	-----------------------------

6. Responda:

- Com quantas threads você obteve o melhor tempo? A eficiência já estava caindo antes de você esgotar os núcleos disponíveis? Se sim, qual característica da sua máquina explica isso (núcleos de desempenho e eficiência diferentes, hyperthreading, limitação de memória, poucos núcleos físicos)? Use `lscpu` se precisar confirmar.
- Os dois resultados (sequencial e paralelo) deram o mesmo número de primos em todas as contagens de threads? Deveriam dar sempre — por quê, neste programa em particular, não há corrida de dados possível mesmo sem nenhuma proteção além do `reduction`?

Erros comuns

Sintoma	Causa provável
<code>undefined reference to 'omp_get_thread_num'</code> (ou similar)	esqueceu <code>-fopenmp</code> na compilação (o <code>make run-<nome></code> já inclui a flag; só falta se você compilar manualmente)
O programa roda sequencial mesmo com o <code>pragma</code> no lugar	o <code>pragma</code> foi ignorado por falta de <code>-fopenmp</code> , e as chamadas de runtime deram erro de ligação — releia a Parte 1, item 4
<code>error: 'x' is not specified in enclosing 'parallel'</code> Speed-up muito abaixo do esperado	<code>default(none)</code> cobrando uma variável que falta em <code>shared(...)</code> <code>OMP_NUM_THREADS</code> não foi definido, ou é menor que o número de núcleos livres

Entrega

- O código final dos dois programas (`ola_mundo_threads.c`, `conta_primos.c`, ambos em `codigo/src/`).
- A tabela de speed-up e eficiência da Parte 3.
- Um parágrafo por pergunta feita ao longo do roteiro.